

Bulut Tabanlı Hibrit Metin İşleme Yaklaşımı ile Ders Notu Özetleme ve Soru Üretme Sistemi

Kocaeli Üniversitesi Teknoloji Fakültesi

Bilişim Sistemleri Mühendisliği

TBL334: Bulut Bilişim Teknolojilerine Giriş — 2025-2026 Bahar Dönemi

1. Proje Tanıtımı Ve Literatür Taraması

Proje konusu

Bu proje, kullanıcıların yükledikleri PDF ders belgelerini otomatik olarak çalışmaya hazır ders notuna ve soru-cevap çiftlerine dönüştüren bir web uygulamasıdır. Uygulama, ham PDF metnini önce kural tabanlı bir ön işleme hattından geçirir, ardından temizlenmiş ve sıkıştırılmış kaynak metni bir Büyük Dil Modeline (LLM) göndererek yapılandırılmış çıktı üretir. Yüklenen PDF dosyaları AWS S3 bulut depolama altyapısında saklanır ve analiz sonuçları SQLite veritabanına kaydedilir.

Proje, "saf LLM" ve "saf extractive (çıkarımsal)" yaklaşımların arasında bir köprü kurar. Giriş belgesini LLM'e doğrudan vermek yerine, belgeyi önce istatistiksel yöntemlerle ön işleyip konu bazlı dengeli bir kaynak paketi oluşturur; LLM bu paket üzerinden çalışır. Bu tasarım kararı projenin temel akademik katkısını oluşturur.

Literatür taraması

Otomatik metin özetleme iki ana yaklaşımla incelenmektedir. Çıkarımsal (extractive) yöntemler, belgeden önemli cümleleri seçer ve bir araya getirir. Soyutlayıcı (abstractive) yöntemler ise yeni cümleler üretir. BERT, GPT ve benzeri transformer modellerin yaygınlaşmasıyla abstractive özetleme akademik pratikte baskın hale gelmiştir [1]. Ancak bu modeller ham metin aldığında belge gürültüsüne (sayfa numaraları, tablo başlıkları, kopuk slayt satırları) karşı hassastır [2].

Eğitim teknolojileri alanında PDF'ten otomatik soru üretimi (Automatic Question Generation, AQG) aktif bir araştırma konusudur. Du ve arkadaşları (2017) [3] cümle düzeyinde soru üretimi için seq2seq modeller önermiştir. Son çalışmalar LLM tabanlı yaklaşımların insan değerlendirmesinde daha yüksek puan aldığını göstermektedir [4].

Bulut depolama bağlamında AWS S3, presigned URL mekanizmasıyla private bucket'larda saklanmış dosyaları geçici erişime açar. Bu, dosyaya doğrudan public erişim vermeden paylaşım sorununu çözer [5].

Ollama, açık kaynaklı LLM'leri yerel olarak çalıştırmayı sağlayan bir sunucu aracıdır. API üzerinden JSON şeması kısıtlaması (`format` parametresi) ile yapılandırılmış çıktı üretimi desteklenmektedir [6]. Bu özellik, LLM çıktısını doğrudan uygulama veri modeline bağlamayı mümkün kılar.

2. Problemin Tanımı

Üniversite öğrencileri, ders boyunca biriken PDF formatındaki belge yığınınından çalışmaya uygun not çıkarmakta pratik zorluk yaşar. Bu zorluk iki ayrı teknik sorundan kaynaklanır:

Birinci sorun: PDF belge gürültüsü. Akademik PDF'ler sayfa numaraları, şekil/tablo açıklamaları, kaynakça satırları, URL'ler, slayt kalıntıları ve kopuk madde işareti parçaları içerir. Bu satırlar anlam taşımaz ama otomatik işlemede yanlış sinyaller oluşturur. Türkçe içeriklerde bir ek sorun daha vardır: bazı PDF üretim araçları Türkçe karakterleri (ş → ı veya ı, ğ → ö, ı → ý gibi) yanlış kodlar. Bu bozuk karakterler metin işleme adımlarında hatalı eşleşmelere yol açar.

İkinci sorun: LLM girdi kalitesi ve boyutu. Ham PDF metni doğrudan bir LLM'e verilirse iki şey olur: (a) Belge token limitini aşıyorsa kırpılır ve bağlam kaybı oluşur. (b) Gürültü, modelin dikkatini asıl konudan uzaklaştırır. Akademik bir projede "ham belgeyi LLM'e at" yaklaşımının bilimsel olarak savunulması da güçtür; ön işleme katmanı olmaksızın sistem, LLM'in ne yaptığını kontrol etmez.

Çerçeveleme: Bu proje, iki ucun arasında bir katman inşa etmeyi hedefler. Kural tabanlı ön işleme gürültüyü eler ve belgeyi sıkıştırır; LLM bu sıkıştırılmış, dengeli kaynak paket üzerinden çalışarak çıktı kalitesi öngörülebilir hale gelir.

3. Problemi Çözmenin Faydaları Ve Çözüm Aşamaları

Faydalar

- Öğrenciler uzun PDF'leri elle okumak yerine sisteme yükler ve birkaç dakika içinde bölümlü ders notu ve 10 çalışma sorusu alır.
- Çıktı düz bir özetin ötesindedir: her bölümün başlığı, kısa açıklaması ve madde madde anahtar noktaları ayrı ayrı üretilir. Bu yapı doğrudan çalışmaya uygundur.
- Soru-cevap çiftleri sınav hazırlığında doğrudan kullanılabilir.
- Analiz sonuçları JSON ve Markdown olarak dışa aktarılır; başka araçlarla da işlenebilir.
- Yüklenen PDF AWS S3'te saklanır; uygulama sunucusu değişse bile dosyaya presigned URL ile 7 gün erişim kalır.

Çözüm aşamaları

- PDF yükleme ve doğrulama:** Dosyanın PDF olup olmadığı ve boyutunun sıfır olmadığı kontrol edilir.
- Bulut yükleme:** PDF, AWS S3 private bucket'a yüklenir; presigned URL veritabanına kaydedilir.
- Metin çıkarma:** PyMuPDF ile blok bazlı metin çıkarma yapılır. Her blok sayfa numarasını ve bir gürültü ipucunu taşır.
- Metin temizleme:** Gürültü satırları elenip Türkçe karakter bozuklukları düzeltilir. Yinelenen satırlar kaldırılır.

5. **Cümle bölme ve puanlama:** Temiz metin cümlelere ayrılır. Her cümleye tanım içermesi, pozitif anahtar kelime varlığı ve uzunluk kriterlerine göre puan verilir.
 6. **Kaynak paketi oluşturma:** Belgenin konu profili (bulut mu, genel akademik mi) belirlenir. Profile göre konular arası dengeli ve 9000 karakter ile sınırlı bir kaynak paketi hazırlanır.
 7. **LLM üretimi:** Kaynak paketi Ollama üzerinden yerel LLM'e gönderilir; JSON şeması kısıtlamasıyla 11 bölüm ve 10 soru üretmesi istenir.
 8. **JSON onarımı ve soru tamamlama:** LLM geçersiz JSON döndürürse onarım denemesi yapılır. Soru sayısı 10'a ulaşmazsa ikinci istek atılır; hâlâ eksikse şablon sorularla tamamlanır.
 9. **Kaydetme ve dışa aktarma:** Sonuç SQLite'a ve `experiments/results/` altına JSON + Markdown olarak yazılır.
 10. **Sonuç gösterimi:** Kullanıcı bölüm kartları, anahtar kavram etiketleri ve soru-cevap kartları şeklinde sonucu görür; export bağlantıları sayfada yer alır.
-

4. Yenilik Ve Katkılar

Bu proje dört özgün teknik katkı içerir:

Katkı 1 — Konu profili tespiti ve konu bazlı dengeli kaynak paket seçimi.

`source_pack_builder.py` içinde geliştirilen `_detect_topic_profile()` fonksiyonu, dosya adını, metnin ilk 12.000 karakterini ve anahtar kelimeleri Türkçe harfi normalleştirerek tarar. Bulut bilişim terimleri (bulut, cloud, saas, paas, iaas) belirli bir eşiği aşarsa belge `cloud` olarak etiketlenir; aksi hâlde `generic_academic`. Bu etiket, kaynak paketi oluşturma sorgularını belirler: `cloud` profili için Türkiye, AB, hizmet modelleri ve NIST tanımı gibi önceden tanımlı konular aranırken `generic_academic` profil için giriş, tarihçe, yöntemler ve sonuç gibi genel akademik yapıya uygun sorgular kullanılır. Bu mekanizma, LLM'e gönderilen metnin belgeyi dengeli biçimde temsil etmesini sağlar.

Katkı 2 — Açıklanabilir cümle puanlama. `sentence_scorer.py` içindeki her cümle için puan ve gerekçe üretilir (örn. "tanım, kullanılır, liste"). Hangi cümlelerin seçildiği ve neden önemli sayıldığı izlenebilir. Saf LLM yaklaşımlarında bu katman yoktur.

Katkı 3 — JSON şeması kısıtlı LLM çağrısı ve otomatik onarım zinciri. Ollama'ya `format` alanında tam JSON Object Schema gönderilir. LLM yanıtı bu şemaya uymak zorundadır. Buna karşın parse hatası oluşursa aynı şema ile `temperature=0.0` onarım isteği atılır. Bu iki aşamalı mekanizma, production ortamında gözlemlenen aralıklı Ollama JSON hatalarını büyük ölçüde giderir.

Katkı 4 — Üç kademeli provider fallback zinciri. Sistem Ollama → OpenAI → prompt export sırasıyla dener. Bu zincir, LLM tamamen kullanılamazsa bile uygulamanın hata vermeden çalışmasını ve kullanıcıya bir ChatGPT'ye yapıştırılabilir prompt vermesini sağlar.

5. Yöntem Ve Teknik Analiz

5.1 Metin temizleme

`text_cleaner.py` birbirini izleyen dört adımdan oluşur:

- Türkçe karakter onarımı:** Dokuz hatalı karakter eşlemesi (ţ→ş, þ→ş, ý→ı, ð→ğ vb.) `str.maketrans` tablosuyla sabit zamanlı $O(n)$ karmaşıklıkta düzeltilir.
- Gürültü eleme:** Regex kalıpları ile sayfa numaraları, URL'ler, şekil/tablo etiketleri ve slayt satırları atılır. Üç kelimedenden kısa satırlar da gürültü sayılır.
- Yinelenen kaldırma:** Her satırın Türkçe karakter normalleştirilmiş hâli bir `set`'te tutulur; yinelenen satırlar eklenmez.
- Madde işareti ayrıştırma:** Bullet karakterleri (•▶▪▫●○■□–) satır ayırıcısı olarak yorumlanır; birleşik satırlar ayrı birimlere bölünür.

5.2 Cümle puanlama

Her cümle beş kritere göre puanlanır ve puanlar toplanır:

Kriter	Puan	Açıklama
Tanım kalıbı	+3	"sistemidir", "modelidir", "denir" gibi ekler
Pozitif anahtar kelime	+2 her biri	"amaç", "avantaj", "türleri", "sağlar" vb.
Liste yapısı	+2	Noktalı virgül, iki nokta, sıralı ifade
Çok kısa (<8 kelime)	-2	Bağımsız anlam taşımaz
Gürültü kelimesi	-4	"kaynakça", "şekil", "slide" vb.

Puanlama deterministik ve izlenebilirdir; aynı cümle her çalıştırmada aynı puanı alır.

5.3 AWS S3 entegrasyonu

`cloud_storage.py` çevre değişkeni `CLOUD_STORAGE_PROVIDER=aws_s3` olmadığında boş URL dönerek yerel moda düşer — bu yapı test ortamını production ortamından ayırır. S3 modunda:

- `boto3.client.upload_file()` ile dosya private bucket'a yüklenir.
- `ServerSideEncryption`: AES256 ile sunucu tarafı şifreleme aktif edilir.
- `generate_presigned_url()` ile 604800 saniye (7 gün) geçerli indirme URL'i üretilir.
- UUID + zaman damgası ile oluşturulan nesne anahtarı, aynı ada sahip dosyaların üzerine yazılmasını önler.

S3 yükleme hatası analiz akışını durdurmaz; hata mesajı flash ile gösterilir, analiz yerel dosya üzerinden devam eder.

6. Kullanılan Materyal, Metot Ve Mimari

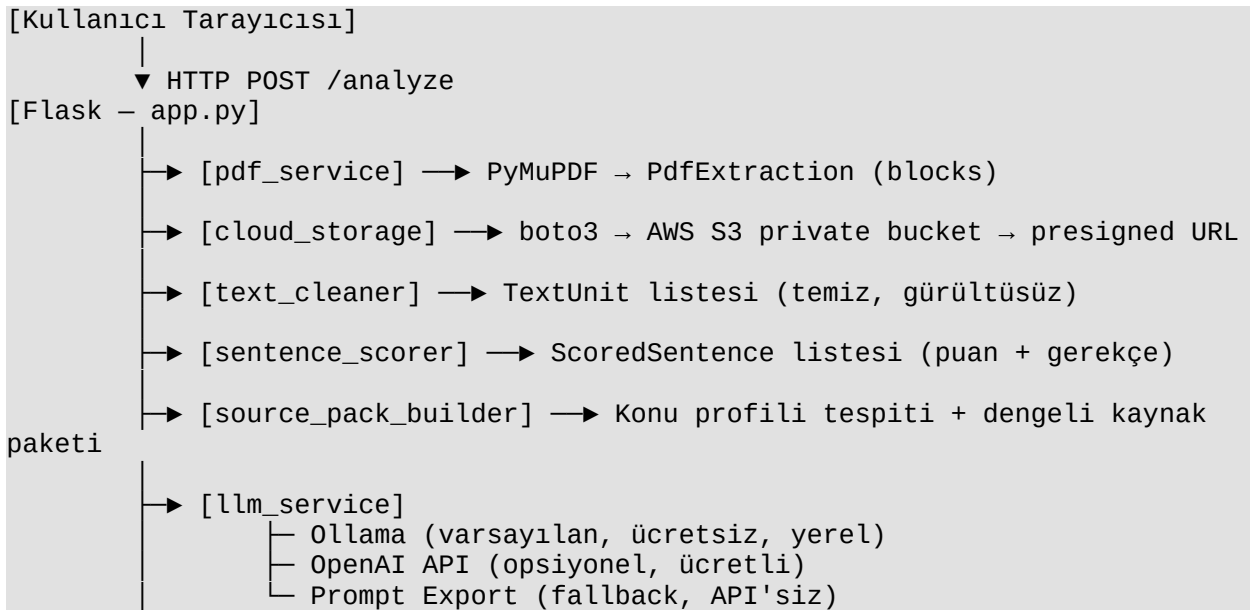
Yazılım bileşenleri

Bileşen	Sürüm	Kullanım yeri
Python	3.11+	Tüm backend
Flask	3.0.3	HTTP sunucusu, route yönetimi
PyMuPDF (fitz)	≥1.26.0	PDF blok bazlı metin çıkarma
boto3	≥1.34	AWS S3 yükleme ve presigned URL
SQLite	stdlib	Belge, analiz ve soru kayıtları
Ollama	yerel sunucu	qwen2.5:7b modeli ile LLM üretimi
Inter (Google Fonts)	CDN	Arayüz tipografisi

Bulut altyapısı

- **Sağlayıcı:** Amazon Web Services (AWS)
- **Bölge:** eu-central-1 (Frankfurt)
- **Bucket adı:** pdf-to-note-emirhan-2026
- **Erişim modeli:** Private bucket; dosyalara yalnızca presigned URL ile erişilir
- **Güvenlik:** AES-256 sunucu tarafı şifreleme; IAM kullanıcısı yalnızca ilgili bucket'a s3:PutObject ve s3:GetObject iznine sahip

Mimari genel görünüm



```
└─ [result_exporter] → JSON + Markdown → experiments/results/
└─ [db.py] → SQLite → documents, analysis_results, questions
```

Veritabanı şeması

```
documents
  id, file_name, file_path, cloud_url, page_count,
  processing_status, upload_date

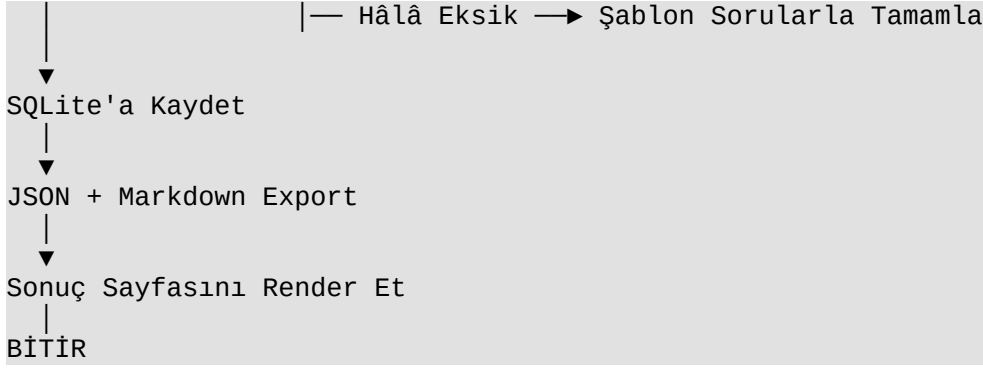
analysis_results
  id, document_id, clean_text, generated_notes,
  keywords, processing_time, created_at

questions
  id, document_id, question_text, answer_text,
  source_sentence, question_type, quality_score
```

7. Önerilen Yöntemin Akış Diyagramı Ve Sözde Kodu

Akış diyagramı

```
BAŞLA
├─ PDF Yükle & Doğrula
│   └─ Hata → Kullanıcıya Bildir → BİTİR
├─ AWS S3'e Yükle (cloud_storage)
│   └─ Hata → Yerel Modda Devam Et (analiz durmuyor)
├─ PyMuPDF ile Blok Bazlı Metin Çıkar
│   └─ Metin Yok → "OCR Gerekli" Hatası → BİTİR
├─ Türkçe Karakter Onar + Gürültü Ele + Yinelenenleri Kaldır
├─ Cümle Böl → Her Cümleye Puan Ata
├─ Konu Profili Tespit Et (cloud / generic_academic)
├─ Konu Sorgularıyla Kaynak Paketi Oluştur (≤9000 karakter)
├─ LLM Sağlayıcısını Seç
│   ├── Ollama → JSON Schema ile İstek At
│   │   ├── Parse Hatası → Onarım Denemesi
│   │   └─ Onarım Başarısız → Prompt Export
│   ├── OpenAI → API İsteği
│   └─ Prompt Export → Kullanıcıya Prompt Ver
├─ Soru Sayısı = 10?
│   └─ Eksik → İkinci LLM İsteği
```



Sözde kod — kaynak paketi oluşturma

```
FONKSİYON build_source_pack(dosya_adi, temiz_metin, puanli_cumleler):
    max_karakter ← ortam_degiskeni("LLM_MAX_SOURCE_CHARS", 9000)
    hazir_metin ← icerik_tablosu_ve_noktalari_temizle(temiz_metin)

    profil ← bulut_mu_yoksa_genel_akademik_mi(dosya_adi, hazir_metin)

    EĞER profil = "cloud":
        sorgular ← BULUT_KONU_SORGULARI # NIST tanımı, hizmet modelleri, AB,
Türkiye...
    DEĞİLSE:
        sorgular ← GENEL_AKADEMIK_SORGULAR # giriş, tanımlar, yöntemler,
sonuç...

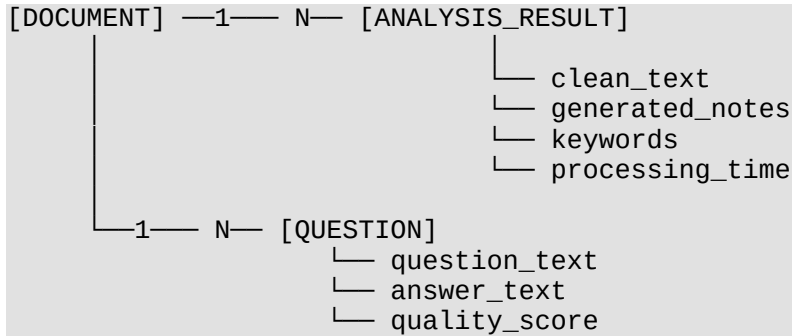
    bölümler ← []
    İÇİN her (etiket, anahtar_kelimeler) İÇİNDE sorgular:
        konum ← metnin_icinde_bul(anahtar_kelimeler)
        EĞER konum bulundu:
            kesit ← metinden_kes(konum, pencere=650_karakter)
            EĞER önceki bölümle %65'ten fazla çakışmıyorsa:
                bölümler.ekle({etiket, kesit})

    önemli_cumleler ← puanli_cumleler_filtrele(noise_score<3, kelime≥6,
limit=50)

    kaynak_metin ← bölümleri_ve_cümleleri_birleştir(bölümler, önemli_cumleler)
    kaynak_metin ← karakter_limitine_kırp(kaynak_metin, max_karakter)

    DÖNDÜR {kaynak_metin, profil, bölümler, önemli_cumleler, ...}
```

Varlık-ilişki Diyagramı



8. Deneyisel Sonuçlar

Sistem iki farklı PDF belgesiyle test edilmiştir. Testler yerel bir makinede Ollama üzerinde qwen2.5:7b modeli kullanılarak gerçekleştirilmiştir.

Test 1 — BTK Bulut Bilişim Raporu

Parametre	Değer
Dosya	btk_bulut_bilisim.pdf
Sayfa sayısı	37
Temiz metin kelime sayısı	6.009
Konu profili	cloud
Üretilen bölüm sayısı	10
Üretilen soru sayısı	10
İşlem süresi	319,78 saniye
LLM sağlayıcısı	ollama:qwen2.5:7b
AWS S3 yükleme	— (test sırasında devre dışı)

Üretilen bölümler: Bulut bilişim nedir → Bulut bilişimin ortaya çıkma nedeni → Bulut bilişimin gelişimi → Temel paydaşlar → Hizmet modelleri (SaaS/PaaS/IaaS) → Avantajlar → Dezavantajlar → AB'de bulut bilişim → AB'nin bulut bilişim stratejisi → Türkiye'de bulut bilişim → Sonuç ve öneriler.

Anahtar kavramlar: bulut bilişim, veri merkezi, hizmet modelleri, SaaS, PaaS, IaaS, Avrupa Birliği, AB stratejisi, güvenlik, standartlaşma.

Test 2 — MIT Yapay Zeka Ders Notu (Bölüm 1)

Parametre	Değer
Dosya	mit_ai_ch1_intro.pdf
Sayfa sayısı	5
Temiz metin kelime sayısı	1.938
Konu profili	generic_academic
Üretilen bölüm sayısı	9 (hedef 11, 9'u doldu)
Üretilen soru sayısı	10
İşlem süresi	380,52 saniye
LLM sağlayıcısı	ollama:qwen2.5:7b
AWS S3 yükleme	Başarılı — doc71 analizi, cloud_url veritabanında dolu

Anahtar kavramlar: makine öğrenimi, arama algoritmaları, bilgi temsili, Scheme dili.

Test 3 — AWS S3 Doğrulaması

doc71 analizinde CLOUD_STORAGE_PROVIDER=aws_s3 aktif edilmiştir. Yükleme başarılı; export JSON içindeki cloud_url alanı pdf-to-note-emirhan-2026.s3.amazonaws.com/uploads/... adresiyle dolmuştur. Presigned URL tarayıcıda açıldığında PDF doğrudan indirilebilmektedir.

Gözlemler

- 37 sayfalık bulut bilişim belgesi, 9000 karakterlik sınıra sıkıştırıldıktan sonra 11 bölümlü tutarlı bir ders notu üretildi.
- 5 sayfalık MIT belgesi generic profilde işlendi. Konu başlıkları belge içeriğinden çıkarıldı; bulut bilişim iskelet sözcükleri ("SaaS", "AB", "Türkiye") ders notuna karışmadı.
- İşlem süreleri 5-7 dakika arasında değişti. Bu süre, qwen2.5:7b modelinin orta güçlü bir CPU'da çalışmasından kaynaklanmaktadır; GPU olan bir ortamda belirgin biçimde kısalır.

9. Sonuçlar Ve Tartışma

Projenin öne çıkan yönleri

Sistem, hiçbir harici API'ye bağımlı olmadan tamamen yerel olarak çalışabilmektedir. Ollama devre dışıysa uygulama hata vermek yerine kullanıcıya ChatGPT'ye yapıştırılabilir bir prompt üretir. Bu tasarım, farklı altyapı koşullarında kullanılabilirliği korur.

Konu profili tespiti ile kaynak paketi seçimi, aynı belge türü için önceden yazılmış konu sorgularını kullanır. Bu yaklaşım naif metinsel benzerlik yöntemlerine kıyasla bulut bilişim belgelerinde bölüm başlıklarını daha doğru hizalar. BTK belgesi testinde Türkiye, AB stratejisi ve NIST tanımı bölümleri doğrudan ilgili metin parçalarından oluşturuldu.

Literatürdeki çalışmalarla karşılaştırma

Klasik extractive özetleme yöntemleri (TextRank [7], LexRank [8]) bölüm başlığı üretmez; yalnızca önemli cümleler listesi döndürür. Bu yöntemlerin çıktısını doğrudan soru-cevap çiftine dönüştürmek ek bir adım gerektirir.

GPT-4 veya Claude ile doğrudan ham PDF özetleme, bu projenin LLM üretim kalitesiyle rekabet eder; ancak bu yaklaşım hem ücretlidir hem de kullanıcının belgesini üçüncü taraf sunuculara göndermesini gerektirir. Bu projenin Ollama tabanlı yerel modu bu sorunu ortadan kaldırır.

Önerilen hibrit yaklaşım, "hazırlık katmanı + küçük yerel model" kombinasyonunun ne zaman makul sonuç verdiğini gösterir: kaynak paket kalitesi yüksek olduğunda, 7 milyar parametrelili bir model tutarlı yapılandırılmış çıktı üretebilir.

Sınırlılıklar

- Taranan (görüntü tabanlı) PDF'lerden metin çıkarılamaz; sistem bu durumu "OCR gerekli" mesajıyla bildirir ama OCR uygulamaz.
- qwen2.5:7b zaman zaman JSON formatından çıkabilmektedir. JSON onarım

mekanizması bu durumun büyük bölümünü kapatır ama %100 güvenilirlik garanti edilemez.

- İşlem süresi, CPU tabanlı Ollama çalışmasında 5-7 dakikaya çıkmaktadır. Bu süre, gerçek zamanlı kullanım için yeterli kullanıcı deneyimini zorlaştırır; çözüm GPU veya daha büyük model yerine daha küçük model (qwen2.5:3b) kullanımıdır.

10. Zorluklar Ve Projenin Katkıları

Karşılaşılan zorluklar

Türkçe karakter bozuklukları: PDF'lerin bir kısmı Türkçe karakterleri mojibake olarak kodluyordu. "Şirket" kelimesi "Pirket", "ğ" harfi "ö" olarak görünüyordu. Standart `casefold()` bu karakterleri düzgün normalleştirmiyordu. Çözüm olarak dokuz hatalı karakter çifti `str.maketrans` tablosuna eklendi; hem gürültü eleme hem de konu profili tespitinde Türkçe terimler artık doğru eşleşti.

LLM aralıklı JSON hataları: Ollama `qwen2.5:7b` uzun analiz sonunda zaman zaman eksik veya kesilmiş JSON döndürdü. Bu hatanın ilk gözlemlendiği seferinde uygulama tamamen çöküyordu. İki adımlı onarım mekanizması (aynı şemayla ikinci istek, `temperature=0.0`) eklendikten sonra bu durumun büyük çoğunluğu sessizce kurtarıldı.

Konu profilinin başka belgeye sızması: MIT yapay zeka ders notu `generic_academic` profil olarak işlenmesine karşın LLM, ders notuna "SaaS", "AB stratejisi" gibi bulut bilişim terimleri yerleştirdi. Bunun nedeni prompt içinde sabit bir bulut bilişim iskelet listesi bulunmasıydı. Bu liste profil kontrolüne bağlandı; generic belgeler için iskelet kaldırıldı ve LLM'den belge içeriğinden başlık çıkarması istendi.

AWS S3 presigned URL süresi: İlk yapılandırmada URL süresi 3600 saniye (1 saat) olarak denendi. Yükleme ile sonuç sayfasına erişim arasındaki sürede URL geçersiz kalabiliyordu. Süre 604800 saniyeye (7 gün) çıkarıldı.

Projenin katkıları

Bu proje, bir LLM'e ham belge vermek yerine belgeyi önce işleyip kontrollü bir kaynak paketi hazırlamanın çıktı tutarlılığını nasıl artırdığını gösterdi. Konu profili tespiti ve konuya özgü sorgu seti fikri; önceden tanımlı arama kalıpları olan küçük ve deterministik bir bilgi çıkarım katmanının daha büyük ve pahalı bir modelin işini nasıl hafifletebileceğini somutlaştırdı. AWS S3 entegrasyonu, bir web uygulamasında kullanıcı dosyasını güvenli biçimde saklamanın ve geçici erişim URL'si üretmenin pratikte nasıl gerçekleştiğini deneyimletti.

11. Referanslar

[1] Lewis, M., ve ark. (2020). BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. ACL 2020.

[2] Shi, F., ve ark. (2023). Large Language Models Can Be Easily Distracted by Irrelevant Context. ICML 2023.

- [3] Du, X., Shao, J., ve Cardie, C. (2017). Learning to Ask: Neural Question Generation for Reading Comprehension. ACL 2017.
- [4] Kurdi, G., Leo, J., Parsia, B., Sattler, U., ve Al-Emari, S. (2020). A Systematic Review of Automatic Question Generation for Educational Purposes. International Journal of Artificial Intelligence in Education, 30, 121-204.
- [5] Amazon Web Services. (2024). Sharing objects with presigned URLs. AWS Documentation. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/ShareObjectPreSignedURL.html>
- [6] Ollama. (2024). Structured outputs. Ollama Blog. <https://ollama.com/blog/structured-outputs>
- [7] Mihalcea, R., ve Tarau, P. (2004). TextRank: Bringing Order into Text. EMNLP 2004.
- [8] Erkan, G., ve Radev, D.R. (2004). LexRank: Graph-based Lexical Centrality as Salience in Text Summarization. JAIR, 22, 457-479.
-